



Memory Allocation Assignment

Ain Shams University — Faculty of Engineering
CSE 335s — Operating Systems

Name: Arsany Hany Anwar

Section: 4

ID: 2200655

Code Files on [GitHub Repo](#)

[Program exe file](#)

Contents

1. Introduction	1
2. System Architecture.....	2
3. Code Walkthrough	3
3.1 Segment class.....	3
3.2 Memory.....	3
3.3 Allocation	4
3.4 Deallocation	4
3.5 Hole selection algorithms	5
3.6 GUI (gui.py)	6
4. Test Cases.....	7
Initial memory setup.....	7
4.1 First-Fit	7
4.2 Best-Fit	13
5. Conclusion.....	17

1. Introduction

Segmentation is a memory-management scheme that supports the user view of a program. Rather than treating memory as a flat array of bytes, segmentation divides a process into logical units called segments, e.g., Code, Data, Stack, etc., each with its own size and base address.

This assignment implements dynamic memory allocation and deallocation using segmentation, with two hole-selection strategies:

- **First-Fit:** allocates the first hole large enough to hold the segment.
- **Best-Fit:** allocates the smallest hole that is still large enough, minimizing leftover fragment.

After each operation the system displays a memory layout drawing and a segment table for every allocated process.

2. System Architecture

The application is structured as a layered class hierarchy. Each class has a single clear responsibility.

Class	Responsibility
Segment (ABC)	Abstract base for all memory blocks. Stores size, name, and starting address.
Hole	A free region of memory. Can grow or shrink as segments are allocated and freed.
InvalidBlock	An occupied region not belonging to any process and can't be allocated.
SegmentOfProcess	A segment that belongs to a process. Carries a data-type label (Code, Data, Stack).
Process	A collection of SegmentOfProcess objects. Tracks total size and segment list.
Memory	Owns the memory block dict, holes list, and starting-address index. Handles merging and gap detection.
Allocator (ABC)	Abstract base with shared allocate/deallocate logic. Subclasses implement choose_hole.
FirstFitAllocator	Implements choose_hole: returns the first hole $\geq$ segment size.
BestFitAllocator	Implements choose_hole: returns the smallest hole $\geq$ segment size.
MemoryGUI	Tkinter GUI. Renders the memory bar, segment tables, and all control panels.

3. Code Walkthrough

3.1 Segment class

Segment is an abstract base class that all memory-block types inherit from. It stores the three properties common to every block:

```
class Segment(ABC):
    def __init__(self, size=0, name=None, starting_address=0):
        self.size = size
        self.starting_address = starting_address
        self.name = name
        if self.name is None:
            self.__class__._counter += 1
            self.name = f"{self.get_prefix()} {self.__class__._counter}"

    @abstractmethod
    def get_prefix(self): pass    # 'H', 'I', or 'S'
```

Hole adds mutability (set_size, shrink_by, grow_by). InvalidBlock and SegmentOfProcess are leaf classes; SegmentOfProcess also carries a reference to its owning Process, which is critical for identity-based deallocation.

3.2 Memory

Memory maintains three parallel data structures that must always be consistent:

- `_memory_block`: dict mapping `starting_address` → Segment object, which is important for drawing the layout.
- `_holes`: list of Hole objects currently available for allocation.
- `_starting_addresses`: sorted list of all keys in `_memory_block`, used for gap detection.

At initialisation, holes are inserted, then `_initialize_invalid_blocks` fills the gaps between holes and the end of memory:

```
def _initialize_invalid_blocks(self):
    i = 0
    while i < len(self._starting_addresses) - 1:
        current = self._memory_block[self._starting_addresses[i]]
        next_addr = self._starting_addresses[i + 1]
        gap = self._calculate_gap_between_segments(current, next_addr)
        if gap.size > 0:
            self.add_segment(gap)
            self._starting_addresses.append(gap.starting_address)
            self._starting_addresses.sort()
        i += 1
    # Trailing gap: space between last segment and end of memory
    if self._starting_addresses:
        last = self._memory_block[self._starting_addresses[-1]]
        trailing = self.total_size - last.get_ending_address()
        if trailing > 0:
            gap = InvalidBlock(trailing, f"I_{last.starting_address}",
                               last.get_ending_address())
            self.add_segment(gap)
            self._starting_addresses.append(gap.starting_address)
            self._starting_addresses.sort()
```

Note: the trailing-gap check was a bug fix applied during development, where the original loop stopped one entry short, missing the region between the last hole and total_size.

3.3 Allocation

This strategy performs a trial allocation on copies of only the structural containers (dict and holes list), leaving the original segment objects themselves untouched, then adopting these new containers if no errors occur:

```
def allocate(self, process: Process):
    trial_memory_block = dict(self.memory.get_memory_block()) # shallow copy
    trial_holes = [copy.copy(h) for h in self.memory.get_holes()] # copy holes
    only

    for segment in process.get_segments():
        chosen_hole = self.choose_hole(segment, trial_holes)
        if chosen_hole is None:
            print(f"== Process {process.get_name()} does not fit! ==")
            return # live memory untouched

        # Apply to trial state
        seg_addr = chosen_hole.get_starting_address()
        segment.set_starting_address(seg_addr)
        del trial_memory_block[seg_addr]
        chosen_hole.set_starting_address(seg_addr + segment.get_size())
        chosen_hole.shrink_by(segment.get_size())
        if chosen_hole.get_size() > 0:
            trial_memory_block[chosen_hole.get_starting_address()] = chosen_hole
        else:
            trial_holes.remove(chosen_hole)
        trial_memory_block[segment.get_starting_address()] = segment

    # Commit
    self.memory.set_memory_block(dict(sorted(trial_memory_block.items())))
    self.memory.set_holes(trial_holes)
    self.memory._starting_addresses = sorted(trial_memory_block.keys())
    self.memory.add_process_to_list(process)
```

A shallow dict copy shares the existing SegmentOfProcess values by reference. New Hole copies are needed because their size and address are mutated during the trial loop. SegmentOfProcess objects must not be copied so that seg.get_process() identity is preserved.

3.4 Deallocation

Deallocation removes all segments of the chosen process, converts their address ranges into new Hole objects, merges any contiguous holes, then regenerates invalid blocks:

```
def deallocate(self, process: Process):
    # 1. Collect freed ranges
    freed_regions = [
        (seg.get_starting_address(), seg.get_size())
        for seg in self.memory.get_memory_block().values()
        if isinstance(seg, SegmentOfProcess) and seg.get_process() == process
    ]
    # 2. Remove process segments AND all invalid blocks
    addrs_to_remove = [
        addr for addr, seg in self.memory.get_memory_block().items()
        if seg.get_process() == process or seg.get_size() == 0
    ]
    for addr in addrs_to_remove:
        del self.memory.get_memory_block()[addr]
```

```

        if (isinstance(seg, SegmentOfProcess) and seg.get_process() == process)
            or isinstance(seg, InvalidBlock)
    ]
    for addr in addrs_to_remove:
        del self.memory.get_memory_block()[addr]
    self.memory.processes.remove(process)

    # 3. Create holes for freed regions
    for start, size in freed_regions:
        new_hole = Hole(size=size, starting_address=start)
        self.memory.get_memory_block()[start] = new_hole
        self.memory.get_holes().append(new_hole)

    self.memory._merge_contiguous_holes()

    # 5. Sync _memory_block to remove absorbed holes
    live_addrs = {h.get_starting_address() for h in self.memory.get_holes()}
    for addr in list(self.memory.get_memory_block()):
        if isinstance(self.memory.get_memory_block()[addr], Hole)
            and addr not in live_addrs:
            del self.memory.get_memory_block()[addr]

    # 6. Rebuild index and regenerate invalid blocks
    self.memory._starting_addresses = sorted(self.memory.get_memory_block().keys())
    self.memory._initialize_invalid_blocks()
    self.memory._sort_segments_by_address()

```

Invalid blocks are always removed before the freed holes are inserted and before `_merge_contiguous_holes` runs, because they become stale the moment any segment is removed. They are always regenerated from scratch at the end.

3.5 Hole selection algorithms

Both allocators inherit from `Allocator` and override `choose_hole` method.

First-Fit

```

def choose_hole(self, segment, hole_list):
    for hole in hole_list:
        if segment.get_size() <= hole.get_size():
            return hole
    return None

```

Best-Fit

```

def choose_hole(self, segment, hole_list):
    best_hole, best_fragment = None, float('inf')
    for hole in hole_list:
        if hole.get_size() >= segment.get_size():
            fragment = hole.get_size() - segment.get_size()
            if fragment < best_fragment:
                best_fragment, best_hole = fragment, hole
    return best_hole

```

3.6 GUI (gui.py)

The GUI is built with Tkinter and divided into three panels:

- Controls (left) — Memory Setup, Allocate Process, and Deallocate Process panels.
- Memory Layout (centre) — a proportionally scaled canvas drawn from `_memory_block`. Holes are blue, invalid blocks grey, each process gets a distinct colour from a fixed palette.
- Segment Tables (right) — a read-only text widget showing each allocated process with its segments, base addresses, and sizes.

After every operation (Apply Setup, Allocate, Deallocate) `_refresh_display()` is called, which redraws the canvas and rewrites the segment table text from the current Memory state.

The failed-allocation case shows a warning messagebox so the user is notified without needing to check the terminal:

```
if process in self.memory.get_process_list():
    self._assign_color(name)
else:
    messagebox.showwarning(
        "Does Not Fit",
        f"Process {name} does not fit in memory."
    )
self._refresh_display()
```

Tkinter was chosen because it is part of Python's standard library, requiring no additional dependencies, which keeps the any project self-contained and simplifies packaging into a single executable with PyInstaller. It provides all the widgets needed for this assignment: canvas for the memory layout drawing, text widgets for segment tables, combo-boxes for algorithm and process selection, and entry fields for user input, without the overhead of a full GUI framework. Making it an excellent choice for creating a GUI suitable for this assignment

4. Test Cases

The following scenario was run twice — once with First-Fit and once with Best-Fit. Screenshots are taken after each operation.

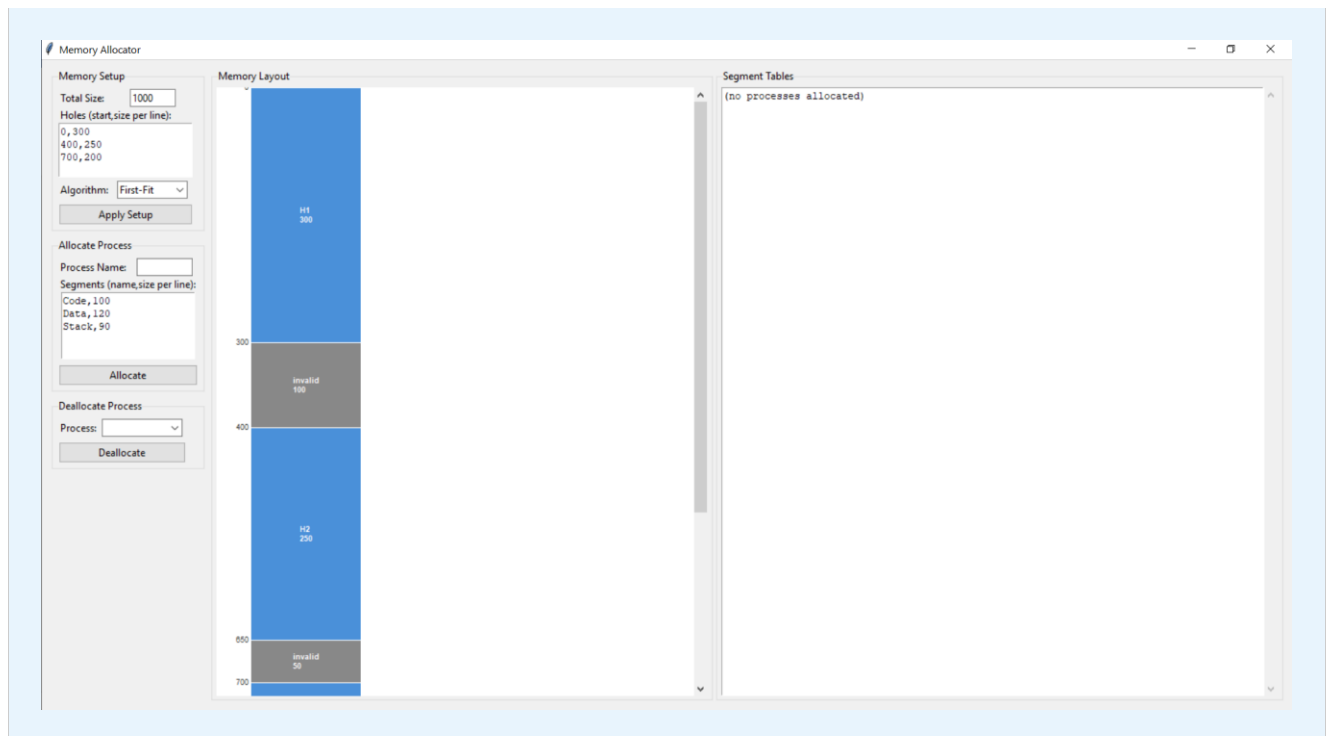
Initial memory setup

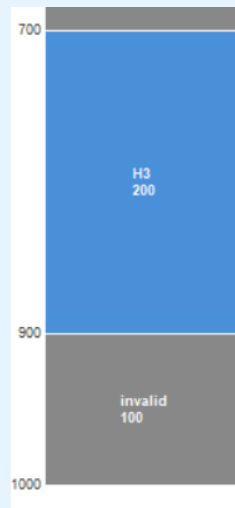
Parameter	Value
Total Memory Size	1000 K
H1	Start = 0, Size = 300 K
H2	Start = 400, Size = 250 K
H3	Start = 700, Size = 200 K

Invalid blocks occupy the non-hole regions: address 300 (size 100) and address 900 (size 100).

4.1 First-Fit

Initial State





First-Fit — Initial memory layout: H1(0,300), InvalidBlock(300,100), H2(400,250), H3(700,200), InvalidBlock(900,100)

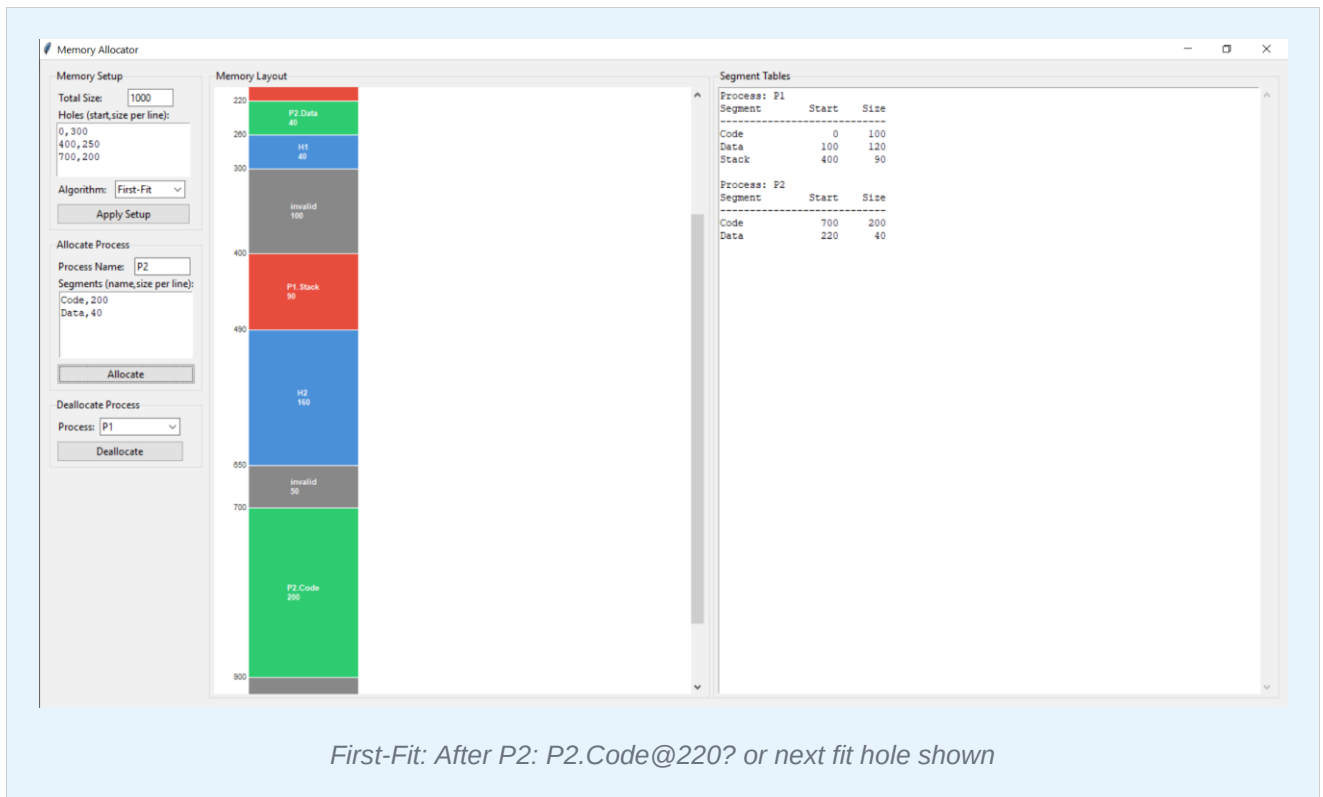
After Allocating P1 (Code=100, Data=120, Stack=90)

First-Fit places Code(100) in H1 at 0, Data(120) at 100, Stack(90). H1 is now too small (80K) so it goes to H2 at 400. H1 shrinks to 80K at address 220, H2 shrinks to 160K at address 490.



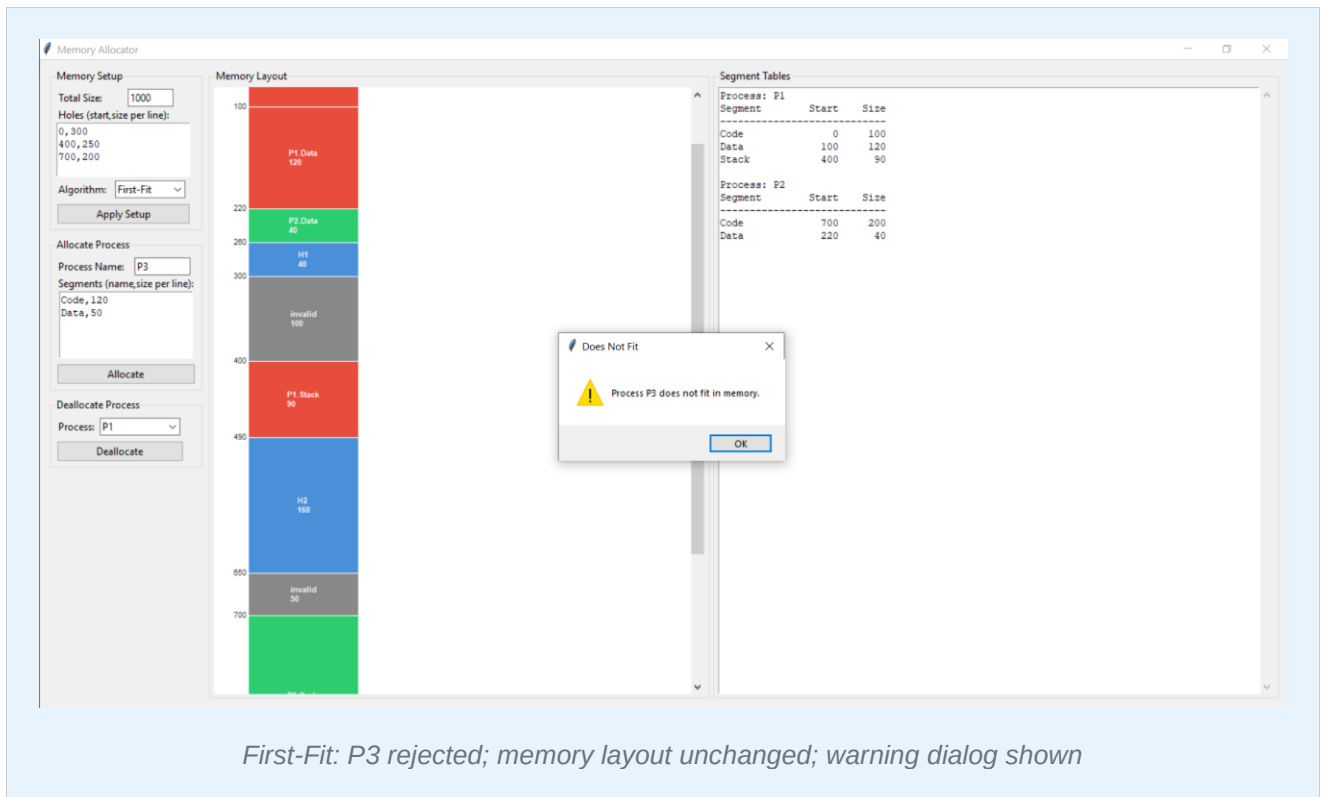
After Allocating P2 (Code=200, Data=40)

For P2.Code(200), H1 remaining (80K) is too small, H2 remaining (160K) is too small, goes to H3(200K) at 700. P2.Data(40) fits in H2 at 490.

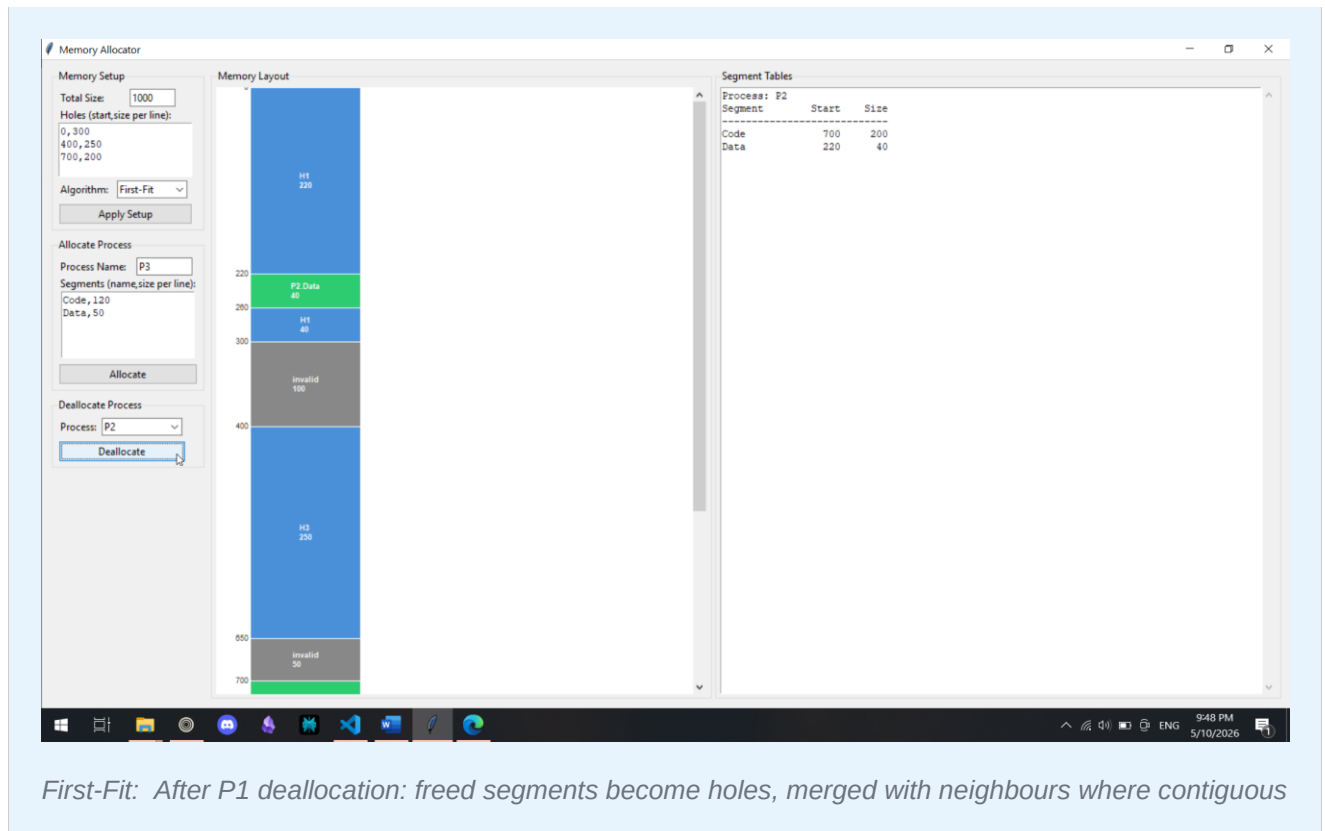


After Attempting P3 (Code=120, Data=50) — Does Not Fit

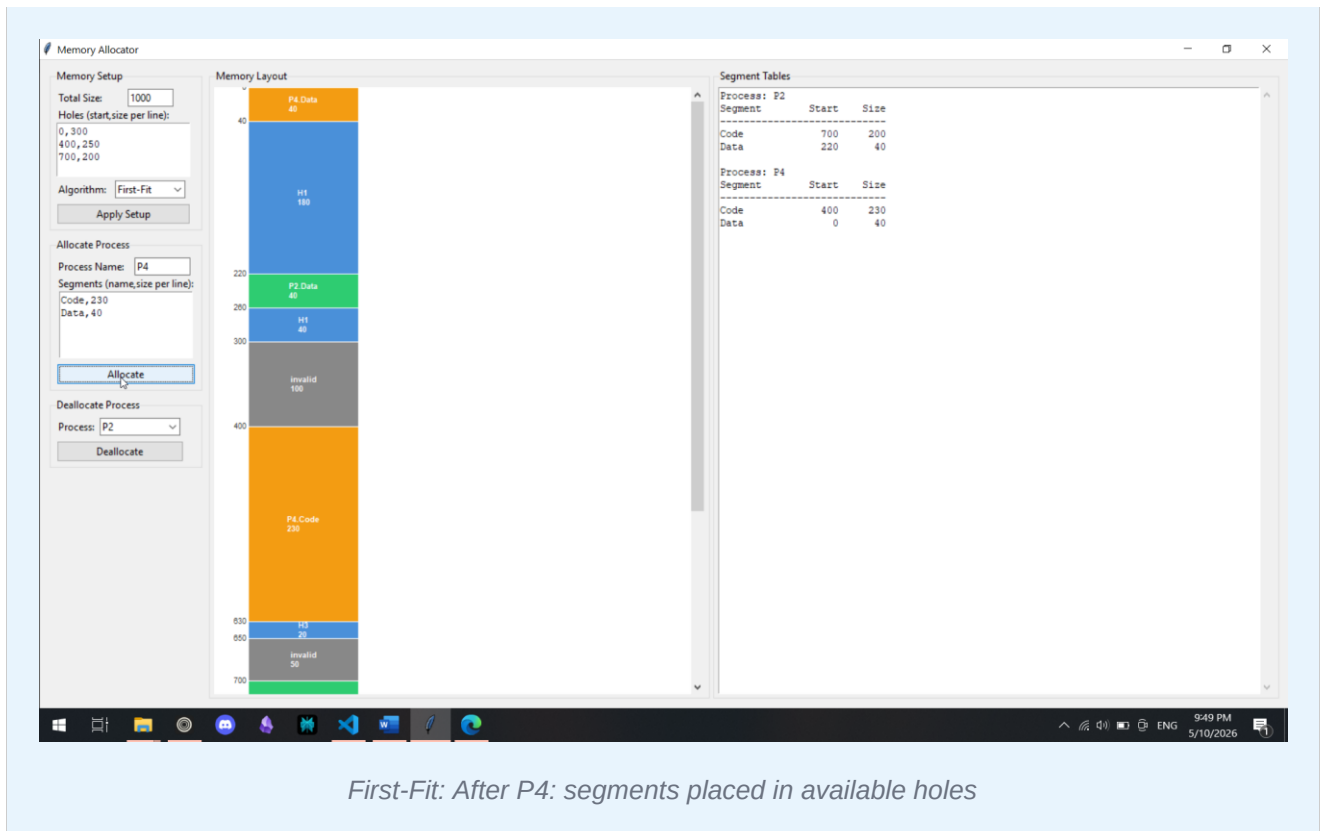
No hole large enough for Code(120) remains. The GUI shows a warning dialog and memory is unchanged.



After Deallocating P1

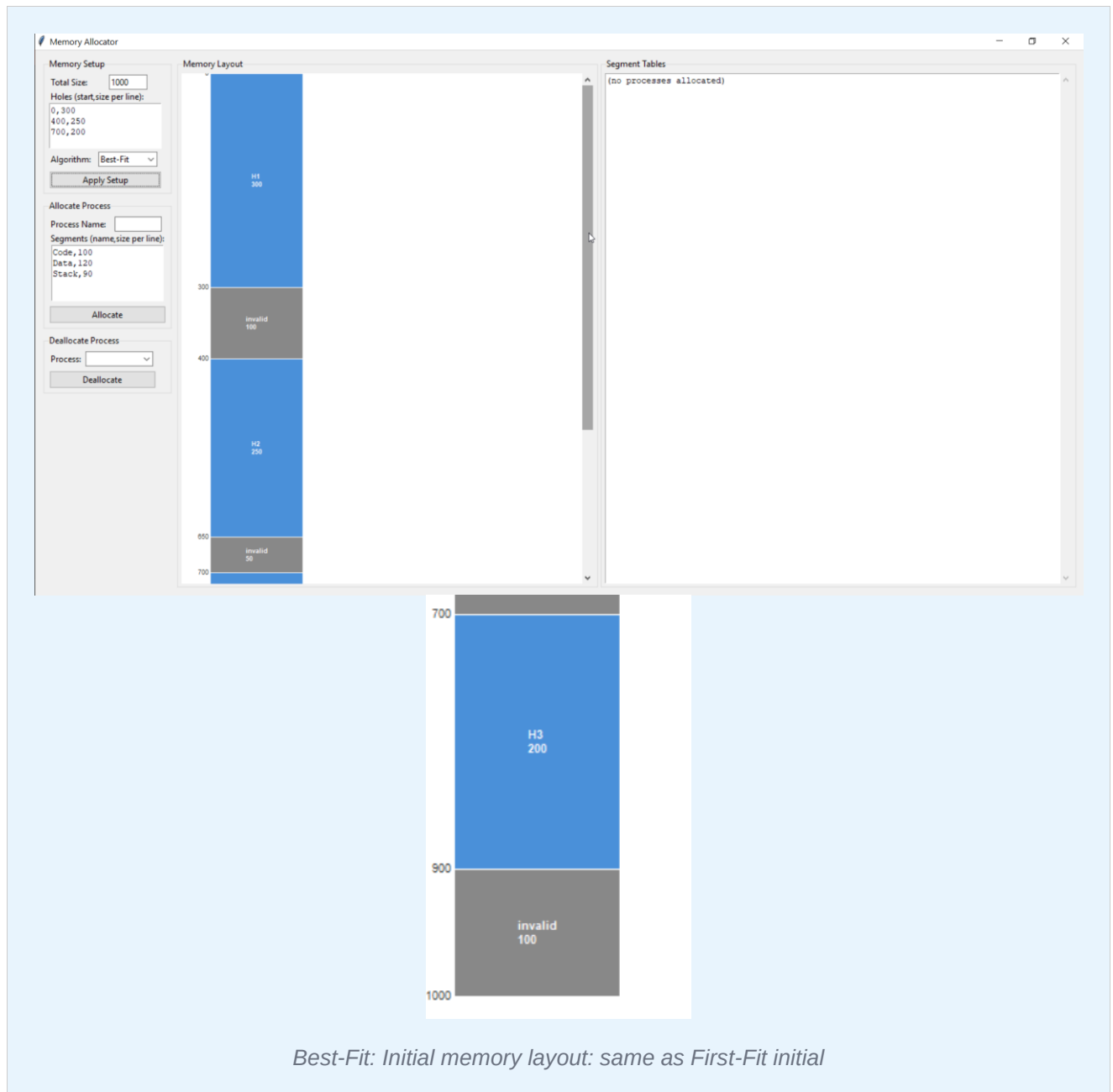


After Allocating P4 (Code=230, Data=40)



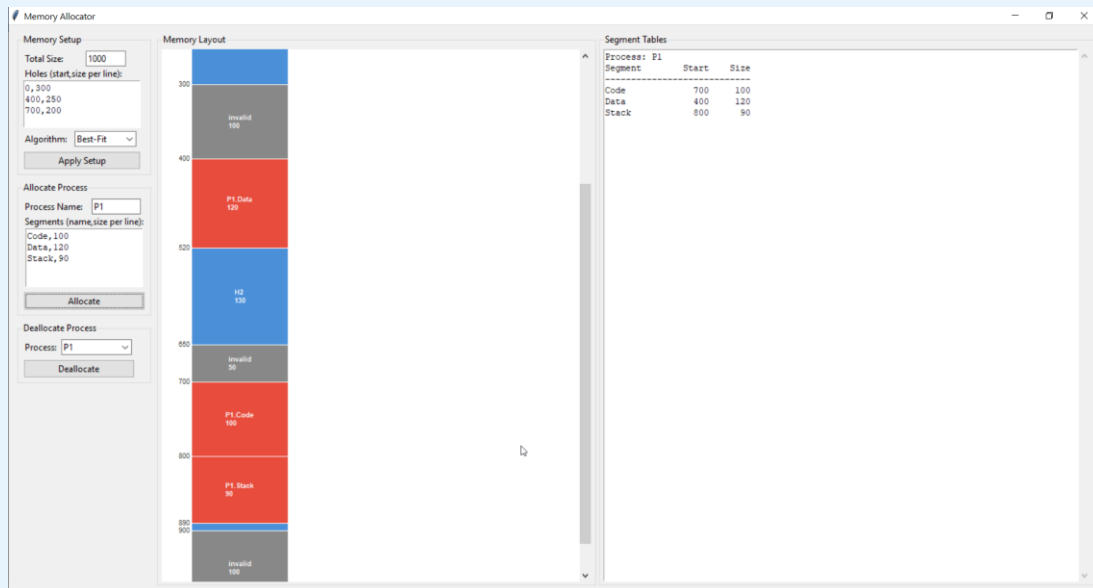
4.2 Best-Fit

Initial State



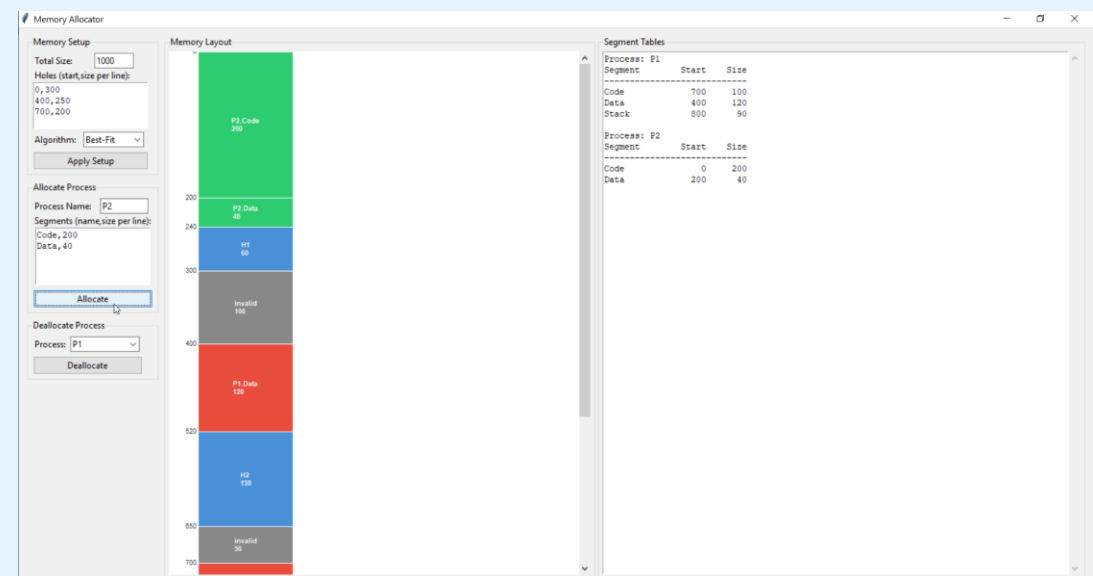
After Allocating P1 (Code=100, Data=120, Stack=90)

Best-Fit picks the hole with the smallest leftover. Code(100): H3(200) gives fragment 100, H1(300) gives 200, H2(250) gives 150; H3 wins. Data(120): H1(300) fragment 180, H2(250) fragment 130; H2 wins. Stack(90): remaining H1(300) fragment 210, remaining H2(130) fragment 40; H2 wins.



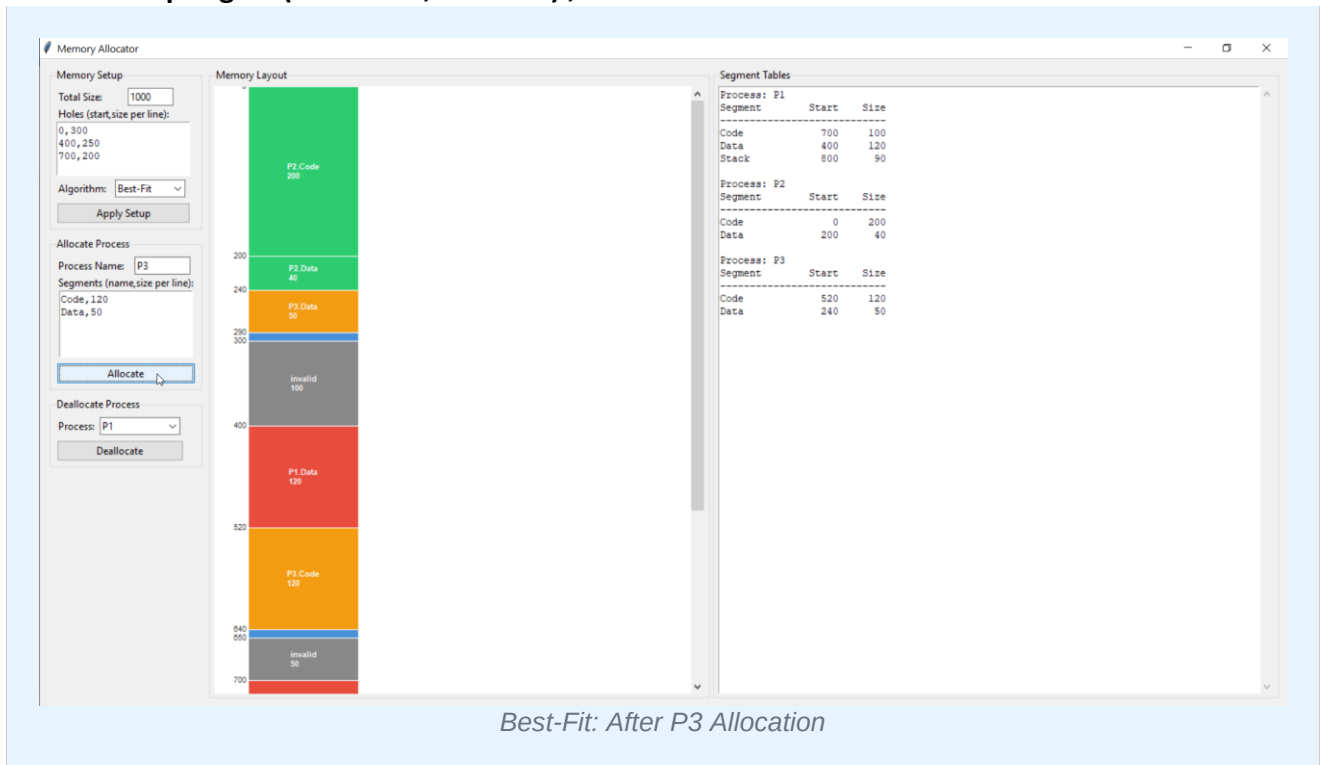
Best-Fit - After P1: Best-Fit selects smallest sufficient hole for each segment

After Allocating P2 (Code=200, Data=40)

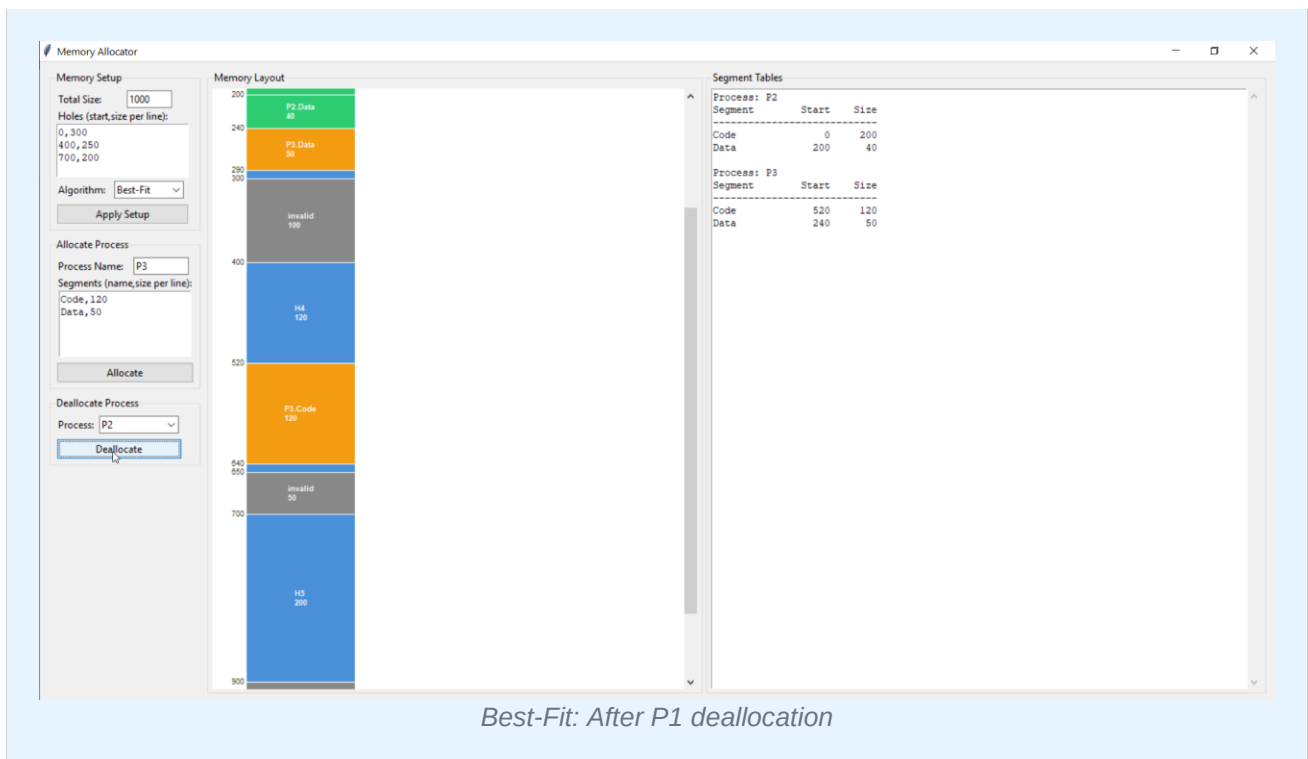


Best-Fit: After P2

After Attempting P3 (Code=120, Data=50) , Does Not Fit



After Deallocating P1



After Allocating P4 (Code=230, Data=40)

The screenshot shows the Memory Allocator application interface. The 'Memory Setup' panel on the left has 'Total Size' set to 1000 and 'Algorithm' set to 'Best-Fit'. The 'Allocate Process' section shows 'Process Name' as 'P4' and 'Segments (name,size per line)' as 'Code, 230' and 'Data, 40'. The 'Allocate' button is highlighted. The 'Memory Layout' panel in the center shows a vertical stack of memory blocks: 'P3 Data 30' (orange, 240-290), 'Invalid 100' (grey, 290-390), 'H4 120' (blue, 400-520), 'P3 Code 130' (orange, 520-650), 'Invalid 30' (grey, 650-680), and 'H5 200' (blue, 680-880). The 'Segment Tables' panel on the right shows the following data:

Process	Segment	Start	Size
Process: P2	Segment		
	Code	0	200
	Data	200	40
	Process: P3	Segment	Start
	Code	520	120
	Data	240	50

A 'Does Not Fit' dialog box is displayed in the center, with a yellow warning icon and the message 'Process P4 does not fit in memory.' and an 'OK' button. The Windows taskbar at the bottom shows the time as 9:56 PM on 5/10/2026.

Best-Fit: P4 does not fit

5. Conclusion

This assignment successfully implemented a memory allocation simulator using segmentation, supporting both First-Fit and Best-Fit algorithms with a functional graphical interface. Through development, several non-trivial bugs were identified and resolved, particularly around memory state consistency during allocation rollback and deallocation, deepening the understanding of reference semantics in Python.

The result is a working tool that accurately reflects the behaviour of dynamic memory allocation, correctly maintaining hole tables, segment tables, and memory layout after every operation.